



Actividad n° 1

Creación de proxy

Integrante: Matias Rojas V.

Profesor: Ivana Bachmann.

Auxiliares: Julián Ferreira.
Cristian Romero V.

Fecha de entrega: 16 de abril de 2026
Santiago, Chile

1. Decisiones de Diseño y Parseo de Mensajes

De acuerdo a lo solicitado, fue necesario evaluar qué información extraer del mensaje HTTP en la función parse y documentarlo.

1.1. Estructura de Datos (Parseo)

Se decidió utilizar una estructura de JSON para almacenar el mensaje HTTP parseado, debido a que en pasos posteriores de la actividad se requiere el uso de archivos JSON para configuraciones y reglas de filtrado. La estructura definida separa los headers del body de la siguiente manera:

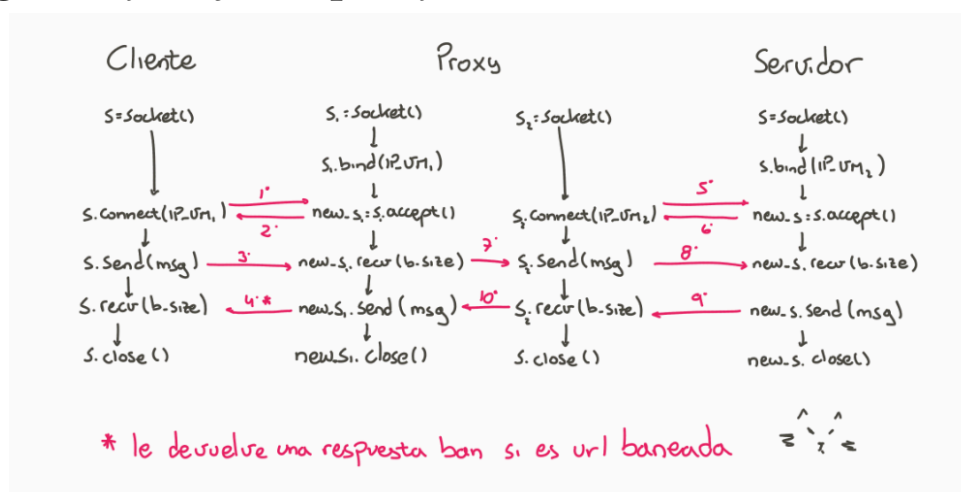
```
{
  "headers": {
    "startLine": "GET / HTTP/1.1",
    "Host": "example.com",
    "headerN": "..."
  },
  "body": "..."
}
```

1.2. Funcionamiento del Código

El servidor proxy funciona recibiendo conexiones de clientes, leyendo la **request** completa y parseándola a json. El flujo principal de funcionamiento es el siguiente:

- El código lee el mensaje JSON y checkea si es un sitio bloqueado.
- Si la URL solicitada se encuentra en la lista de sitios bloqueados, el proxy responde con un código de error 403 y un archivo HTML local (ban.html) que contiene una imagen.
- Si no está bloqueado, el proxy se conecta al servidor destino en el puerto 80, reenvía la request agregando el header X-ElQuePregunta: Jujalag, recibe la response completa, y ejecuta una función para reemplazar palabras prohibidas en el body antes de reenviarlo al cliente.

2. Diagrama y Flujo del proxy



El proxy actúa como intermediario. Utiliza un socket OC servidor asociado a la IP de la máquina virtual (IP_VM) en el puerto 8000 para escuchar peticiones de los clientes (como un navegador o curl).

Cuando un cliente se conecta, el proxy acepta la conexión y lee el requerimiento. Para reenviar esa petición, el proxy crea un **segundo socket** (socket cliente) para conectarse al servidor web destino en el puerto 80.

El proxy envía la request, recibe la respuesta de este servidor a través del segundo socket, cierra la conexión externa, y finalmente devuelve la respuesta procesada al cliente a través del primer socket.

3. Manejo de Buffers y Mensajes HTTP

Al momento de implementar la capacidad de recibir mensajes con buffers más pequeños que el tamaño del mensaje, surgieron interrogantes fundamentales sobre el ciclo de lectura.

¿Cómo sé que el HEAD (las cabeceras) llegó completo?

Se sabe que las cabeceras han terminado de llegar exclusivamente cuando se detecta la secuencia de bytes correspondiente a un doble salto de línea “\r\n\r\n”. El proxy inspecciona el texto acumulado y, en el instante en que encuentra ese patrón, da por terminada la fase de lectura de headers.

¿Qué pasa si los headers no caben en mi buffer?

Si el buffer es más pequeño que el tamaño total de las cabeceras, el proxy no falla ni pierde datos, sino que debe realizar múltiples ciclos de lectura. El sistema lee un fragmento, lo acumula y verifica si el delimitador \r\n\r\n está presente. Si no lo está, vuelve a invocar la función de lectura (recv) hasta completar la secuencia.

¿Cómo sé si el mensaje llegó completo?

Para confirmar que un mensaje se ha recibido al 100%, el proxy lee desde el socket y debe validar dos fases consecutivas:

- Fase de Cabeceras (Headers): Se inicia leyendo el flujo de datos hasta encontrar el delimitador de doble salto de línea (rnrn). Esto indica que la sección de metadatos ha terminado.
- Fase del Cuerpo (Body): Para saber si el body llegó completo, se depende de la información obtenida en la fase anterior, principalmente de la cabecera Content-Length. El sistema sigue leyendo los datos que vienen después del \r\n\r\n hasta que los bytes acumulados coincidan exactamente con el número indicado en dicha cabecera.

4. Pruebas y Experimentos de Buffer

Se realizaron diversas pruebas modificando el tamaño del buffer de recepción (recv_buffer) para garantizar la robustez del código.

4.1. Experimento A: Buffer menor al total, pero mayor a los headers

Configuración: Se ajustó el buffer de lectura a 600 bytes y se envió una petición GET. se observó que la lectura consolidó con una lectura de 143 bytes.

Comportamiento observado: En la primera lectura, el proxy recibió la Start Line y todas las cabeceras (incluyendo \r\n\r\n). El proxy analizó correctamente las cabeceras, identificó el Content-Length, y procedió a realizar múltiples llamadas de lectura en un bucle para consumir los bytes restantes del body.

Resultado: Exitoso. El servidor destino recibió el cuerpo completo sin corrupción de datos.

4.2. Experimento B: Buffer menor a los headers, pero mayor a la Start Line

Configuración: Se configuró el buffer de lectura a 30 bytes. Se envió una petición GET que contenía aproximadamente 143 bytes de cabeceras.

Comportamiento observado: En la primera lectura, el proxy solo recibió la Start Line y el inicio de la primera cabecera. Como no detectó la secuencia `\r\n\r\n`, el proxy almacenó este fragmento en un acumulador y volvió a llamar a `recv`. En las pruebas con buffer de 30, el proceso tomó 5 ciclos (4 lecturas de 30 bytes y una de 23 bytes) antes de consolidar la información y analizar el mensaje completo.

Resultado: Exitoso. El proxy no intentó parsear cabeceras incompletas, acumulando correctamente el estado.

4.3. Experimento C: Buffer Extremadamente Pequeño (Buffer = 6)

Comportamiento observado: Se forzó un buffer de 6 bytes. El servidor requirió 24 lecturas consecutivas (23 lecturas de 6 bytes y 1 de 5 bytes) para terminar de extraer los datos.

Conclusión: Aunque la lectura de múltiples fragmentos funciona teóricamente, usar buffers minúsculos demostró que se corre el riesgo de truncar datos temporalmente, lo que puede causar un parsing inválido si el proxy intenta interpretar la estructura antes de que llegue la secuencia de fin de cabeceras.

4.3.1. Pruebas Adicionales de Integración (Navegador/cURL):

Filtro 403: Se verificó que al intentar acceder a los dominios bloqueados en el archivo `ban.json`, el servidor intercepta la comunicación exitosamente y retorna un código HTML de error en lugar de cargar el sitio destino.

Reemplazo de palabras: Al entrar al sitio de pruebas HTTP permitido por medio del proxy, se confirmó que las palabras definidas como prohibidas en el archivo JSON son interceptadas e inmediatamente reemplazadas por censuras en el cuerpo de la respuesta antes de mostrarse en el navegador local

5. funciones

- `receive_full_msg`: Lee toda la información que llega por un socket hasta asegurarse de tener un mensaje HTTP completo. Para lograrlo, separa las cabeceras (headers) del cuerpo (body) y utiliza el `Content-Length` para saber exactamente cuántos datos debe seguir leyendo.
- `parse_HTTP_msg`: Toma un mensaje HTTP y lo transforma en un json. Esto facilita manipular la información (los headers y body) de forma estructurada.
- `create_HTTP_msg`: Realiza el proceso inverso. Toma un json y lo convierte nuevamente en un mensaje HTTP, actualizando automáticamente el `Content-Length` para que coincida con el tamaño real del cuerpo.
- `read_JSON`: Lee un archivo `.json`
- `read_HTML`: Lee un archivo `.html` (string).
- `read_img`: Lee archivos binarios (imágenes `.jpg`) y devuelve los datos en formato de bytes.
- `check`: Revisa si la URL que el usuario quiere visitar está en la lista de bloqueados. Si la encuentra, bloquea el acceso cambiando la respuesta a un "403 ERROR" y retorna `False`. Si la URL es segura, retorna `True`.
- `forbidden_words`: Escanea el cuerpo del mensaje en busca de palabras prohibidas y las reemplaza por otras palabras o censuras predefinidas, devolviendo el mensaje modificado.